

## Arquivo: node\_service.py

```
"""
DESCRIÇÃO GERAL:
Esta camada representa a lógica de negócio que roda dentro de cada nó de armazenamento.
Ela recebe requisições já direcionadas pelo coordenador e executa as operações
localmente sobre o storage do nó.
"""
```

```
from dfs.protocol import parse_request, make_response
from dfs.storage.local_storage import LocalStorage
```

```
class NodeService:
```

```
    """
```

```
    Serviço de execução local de um nó.
```

```
    """
```

```
    def __init__(self, storage: LocalStorage, node_id: str, shard_id: int):
```

```
        # 0 storage representa o disco local deste nó.
```

```
        self.storage = storage
```

```
        # Identificadores usados para rastreabilidade.
```

```
        self.node_id = node_id
```

```
        self.shard_id = shard_id
```

```
    def dispatch(self, raw_request: bytes) -> bytes:
```

```
        """
```

```
        Processa uma requisição Protobuf e devolve uma resposta Protobuf.
```

```
        """
```

```
        # Converte os bytes da rede em uma estrutura acessível.
```

```
        request = parse_request(raw_request)
```

```
        # Normaliza a operação.
```

```
        op = request.op.upper().strip()
```

```
        try:
```

```
            if op == "PUT":
```

```
                # Grava o arquivo no disco local do nó.
```

```
                self.storage.put(request.path, request.data)
```

```
                return make_response(
```

```
                    True,
```

```
                    "Arquivo salvo com sucesso",
```

```
                    node_id=self.node_id,
```

```
                    shard_id=self.shard_id,
```

```
                )
```

```
            if op == "GET":
```

```
                # Lê o arquivo do storage local.
```

```
                data = self.storage.get(request.path)
```

```
                return make_response(
```

```

        True,
        "Arquivo encontrado",
        data=data,
        node_id=self.node_id,
        shard_id=self.shard_id,
    )

    if op == "DELETE":
        # Remove o arquivo do storage local.
        self.storage.delete(request.path)

        return make_response(
            True,
            "Arquivo removido com sucesso",
            node_id=self.node_id,
            shard_id=self.shard_id,
        )

    if op == "LIST":
        # Lista os arquivos presentes neste nó.
        entries = self.storage.list_files()

        return make_response(
            True,
            "Listagem concluída",
            entries=entries,
            node_id=self.node_id,
            shard_id=self.shard_id,
        )

    # Caso a operação não seja suportada.
    return make_response(
        False,
        "Operação inválida",
        node_id=self.node_id,
        shard_id=self.shard_id,
    )

except Exception as exc:
    # Qualquer erro local é transformado em resposta controlada.
    return make_response(
        False,
        f"Erro: {exc}",
        node_id=self.node_id,
        shard_id=self.shard_id,
    )

```